

Shortest Job First Algorithm | Priority Scheduling | Round Robin CPU Scheduling

Shortest Job First (SJF) — Non-Preemptive

This scheduling method gives the CPU to the process whose Burst Time (BT) is smallest. Once a process starts running, it runs completely — it does not get interrupted midway.

points to remember:

1. **The process with the smallest Burst Time (execution time) runs first.**
2. **We must know or estimate BT of all processes in the Ready Queue**, but estimating exact BT is practically impossible.
3. CPU picks the **job with lowest BT**, runs it fully, then picks the next lowest.
4. **Convoy Effect** occurs if a large BT process arrives first — others have to wait unnecessarily.
5. **Starvation is possible** — long BT processes may get delayed if small tasks keep coming.
6. Processes are considered based on **Arrival Time (AT) + Burst Time (BT)** while ordering.

Example:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	7
P2	2	4
P3	3	1

Working:

- At time 0 → only **P1** is present → it starts executing.
- At time 2 and 3, P2 and P3 arrive, but P1 is already running (non-preemptive).
- After P1 completes, remaining processes are P2 (BT=4) and P3 (BT=1).
CPU chooses **P3 first**, then **P2**.

🕒 Execution Order: **P1 → P3 → P2**

Convoy Effect Example:

Process	AT	BT
P1	0	20
P2	1	3
P3	2	2

P1 (20 units) arrives first, so it will run till the end.

Even though P2 and P3 are much shorter, they wait for a long time → **Convoy Effect**.

Starvation Example:

If small processes keep arriving like:

P1 (BT 1), P2(BT 1), P3(BT 1), P4(BT 1), P5(BT 20)

P5 may wait a very long time → **Starvation**.

2. Shortest Job First — Preemptive (also called SRTF: Shortest Remaining Time First)

Here, CPU always runs the process with the **least remaining burst time**.

If a new process comes with **smaller BT** than the current one, the CPU is immediately switched to it.

Easy understanding:

- Job with **shorter remaining time** gets priority.
- Running process **can be interrupted**.
- Starvation is less than non-preemptive SJF because long tasks eventually finish.
- Convoy effect does not occur since small jobs don't wait behind long ones.
- Average waiting time becomes low, because short jobs finish early.

Example:

Process	AT	BT
P1	0	7
P2	2	4
P3	4	1

Execution Flow

- 0–2 → P1 runs (remaining 5)
- At time 2 → P2 arrives (BT 4 < 5) → preempt
- 2–4 → P2 runs (remaining 2)
- At time 4 → P3 arrives (BT 1 < 2) → preempt
- Then P3 finishes, then P2, then P1 at the end

Order → **P1 → P2 → P3 → P2 → P1**

3. Priority Scheduling — Non-Preemptive

Each process is given a priority when it is created.

Higher priority process runs first.

Easy points:

- Once CPU starts a job, it runs till completion.
- SJF is actually **priority scheduling where short time = high priority**.
- If two jobs have same priority → we use FCFS among them.

Example:

Process	Priority (High=1)	BT
P1	3	5
P2	1	4
P3	2	2

Execution order → **P2 → P3 → P1**

4. Priority Scheduling – Preemptive

Here, running process is stopped if a **higher priority job** arrives.

Points:

- CPU always gives preference to higher priority.
- May cause starvation for low priority process.
- Same starvation issue in both preemptive & non-preemptive.
- **Ageing** solves this → priority increases if a process waits too long.

Simple example of Ageing:

Every 15 minutes increase priority by +1 →

So even a low priority job will eventually reach high enough priority to run.

5. Round Robin Scheduling (RR)

Most widely used in modern OS and especially in time-sharing systems.

Easy logic:

- Works like FCFS, **but with a time limit** (TQ).
- Each process gets a **fixed time slice**.
- If it doesn't finish, it goes back to the queue again.
- Very low starvation – every process gets CPU turn.
- Too small TQ = too many context switches = overhead increases.
- Does not depend on BT, only AT + Time Quantum.

Small example (TQ = 3):

Process	BT
P1	5
P2	4
P3	2

Execution:

- P1 → 3 units (remaining 2)
- P2 → 3 units (remaining 1)
- P3 → 2 units (finishes)

- P1 → 2 units (finishes)
- P2 → 1 unit (finishes)

Order → **P1 → P2 → P3 → P1 → P2**

